Instituto Profesional Virginio Gómez
Universidad de Concepción

De la Arquitectura a la Realidad Física en C#

Lenguaje de Programación OO (0568) – Semana 2

El Umbral del Programa

El punto de entrada donde el Sistema Operativo inicia la ejecución.

Independiente de cualquier instancia (pertenece a la clase base).

No devuelve valores al sistema (por defecto).

```
static void Main(string[] args)
```

La llave maestra que busca el compilador JIT de .NET.

Vectores de comunicación desde la terminal.



PROYECTO: SESIÓN DE 160 MIN
ESCALA: 1:1
FECHA: 2024

Anatomía de una Clase en C#

Abstracción y Encapsulación en su forma más pura.

```
public class Persona
{
    private string _rut;

    public string Nombre { get; set; }

    public void Saludar()
    {
        // Lógica
    }
}
```

Campos Privados

El estado interno. Oculto y protegido.

Propiedades Autoimplementadas

Accesos controlados (`get; set`). C# genera el campo privado invisiblemente.

Métodos

El comportamiento y las acciones disponibles.

	PROYECTO: SESIÓN DE 180 MIN
	ESCALA: 1:1
SS	FECHA: 2024

Campos vs. Propiedades: El Modelo de Seguridad

Protegiendo la integridad de los datos mediante el encapsulamiento.

El Campo Privado



```
private int _edad;
```

Variables crudas. Solo la propia clase puede tocarlas.

La Propiedad



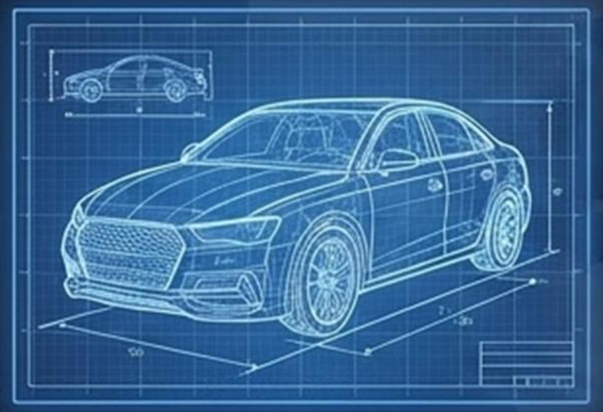

```
public int Edad { get; set; }
```

El contrato público. Intercepta lecturas (get) y escrituras (set) para aplicar validaciones.

PROYECTO- SESIÓN DE 180 MIN
ESCALA: 1.1
FECHA: 2024

La Transición a la Realidad Física

El operador 'new' transforma la idea en materia.



```
class Vehiculo
```

```
new Vehiculo()
```



Al usar 'new', el entorno de ejecución de .NET (CLR) busca espacio físico en la memoria RAM para construir el objeto.

		PROYECTO-SESIÓN DE 180 MIN
		ESCALA: 1:1
		FECHA: 2024

Espacio Físico: Stack vs. Heap

Cómo .NET organiza el almacén de memoria.

El Stack / Pila de Ejecución



Atributos: Rápido, temporal, estructurado (LIFO).

Uso: Almacena variables locales y parámetros de métodos. Se limpia automáticamente al salir del método.

El Heap / Montículo



Atributos: Amplio, dinámico, requiere recolección de basura (Garbage Collector).

Uso: Almacena los objetos instanciados. Necesitamos una 'dirección' (referencia) para encontrarlos.

PROYECTO-SESIÓN DE 180 MIN

ESCALA: 1:1

FECHA: 2024

La Dualidad de Tipos en .NET

Clases (Referencia) vs. Structs (Valor)

	Tipos por Valor (Structs)	Tipos por Referencia (Clases)
Ubicación en Memoria	Viven en el Stack. 	Viven en el Heap (su puntero vive en el Stack). 
Comportamiento al Asignar (=)	Copian el dato completo. 	Copian la dirección de memoria. 
Valor por Defecto	Instancia vacía. 	null. 
Caso de Uso Ideal	Coordenadas, fechas, datos pequeños (Ej: int, DateTime). 	Entidades complejas de negocio (Ej: Persona, Colaborador). 

PROYECTO-SESIÓN DE 180 MIN
ESCALA: 1.1
PECHAÍ 2024

EL EFECTO REFERENCIA: ¿QUÉ OCURRE AL COPIAR?

Entendiendo la asignación: **Objeto A = Objeto B**

(Struct / Valor)



(Clonación física) - Variables totalmente independientes.

(Clase / Referencia)



(Copia de Puntero) - Modificar la TV con el control B afecta lo que ve el control A.

Sintetizando la Creación

IDE

```
Persona p = new Persona();
```



Se crea el puntero (el control remoto) en el Stack.

El puntero se enlaza a la dirección de memoria exacta.

Se construye el objeto físico en el Heap.

		PROYECTO-SESIÓN DE 180 MIN
		ESCALA: 1.1
LS		PECMAÍ 2024

Manos a la Obra: Comprobando la Física

Paso 1: Definiendo las Estructuras de Prueba

Tipo Valor

```
1. public struct Punto
2. {
3.     public int X { get; set; }
4. }
```

Tipo Referencia

```
1. public class Jugador
2. {
3.     public int Puntaje { get; set; }
4. }
```

Nota: Abran su IDE. Creen estas dos estructuras simples. Una es tipo valor, la otra es tipo referencia.

PROYECTO-SESIÓN DE 180 MIN
ESCALA: 1:1
PECHAÍ 2024

Manos a la Obra: Ejecutando el Experimento

Paso 2: Forzar la Copia y Mutar

IDE - Experimento

```
// Experimento Struct
Punto p1 = new Punto { X = 10 };
Punto p2 = p1; // Copia
p2.X = 99;     // Mutación

// Experimento Clase
Jugador j1 = new Jugador { Puntaje = 10 };
Jugador j2 = j1; // Copia
j2.Puntaje = 99; // Mutación
```

¿Qué valor tienen **p1.X** y **j1.Puntaje** ahora?
Discutan con su compañero antes de ejecutar.

PROYECTO-SESIÓN DE T80 MIN
ESCALA: 1.1
PECMAÍ 2024

Resultados del Experimento

El comportamiento dicta la arquitectura.

Terminal Output

```
> Ejecutando...
p1.X = 10
j1.Puntaje = 99
```

El Struct sobrevivió

Al copiar `p2 = p1`, se clonaron los datos físicamente. Mutar p2 no afectó a p1. (Aislamiento seguro de datos).

La Clase mutó

Al copiar `j2 = j1`, solo se copió el puntero. Mutar `j2` alteró la única instancia real compartida en el Heap. (Estado compartido).

Desafío de Ingeniería: Sistema de Recursos Humanos

40 Minutos de Trabajo Individual

Contexto:
Debe diseñar el núcleo de un sistema de control de horas para una organización.

Requerimientos Estrictos:

1. Crear una clase Colaborador (Tipos por Referencia).
2. Crear un struct RegistroHoras (Tipos por Valor).
3. El colaborador debe tener propiedades autoimplementadas para Nombre y Cargo.
4. El colaborador debe incluir un método RegistrarJornada(RegistroHoras registro).



PROYECTO-SESIÓN DE T80 MIN
ESCALA: 1.1
PECMAÍ 2024

Arquitectura Esperada

Diseño esquemático de las entidades.



PROYECTO-SESIÓN DE T80 MIN
ESCALA: 1.1
PECMAÍ 2024

Cierre Metacognitivo

De la teoría a la competencia técnica.



¿Qué construí hoy?

Estructuré clases y structs utilizando propiedades encapsuladas y métodos.

¿Cómo gestiona C# la materia?

Comprendí que la memoria se divide en Stack (rápido, copias físicas) y Heap (referencias, estado compartido).

¿Por qué es crítico?

Un error entre valor y referencia puede corromper los datos de una organización.

Instituto Profesional Virginio Gómez
Universidad de Concepción

PROYECTO-SESIÓN DE T80 MIN
ESCALA: 1:1
PECMAÍ 2024